ChatGPT

Детализированный план реализации промышленной AI content-ops платформы через Claude Code и облако

Executive summary

Ниже — практический план реализации **AI content-ops платформы промышленного уровня**, где чат является не «магическим ботом», а операторским интерфейсом поверх типизированных workflow, очередей, хранилищ, агентов, правил комплаенса и human approval gates. Для такого класса продукта оптимальна **гибридная модель**: интерактивные и жестко контролируемые этапы строятся на **Messages API**, длительные асинхронные исследования и многошаговые фоновые задачи — на **Claude Managed Agents**, а сама инженерная реализация, репозиторий, automation around prompts/config/tests/CI — через **Claude Code**. Такой расклад согласуется с официальным разделением Anthropic между direct model prompting и managed harness for long-running work, а также с тем, что Managed Agents доступны только через прямой Claude API. [1]

Если выбрать один главный архитектурный принцип, то он будет таким: **не строить "один большой чат", а строить версионируемую workflow-платформу**. Ее единицы управления — `Brief`, `ResearchPack`, `CreativeVariant`, `ComplianceVerdict`, `PublishJob`, `RunLog`, schema-validated outputs, citations, eval suites и approval states. Именно это делает систему масштабируемой, проверяемой и пригодной для regulated verticals вроде gambling/affiliate, где требования каналов и юрисдикций нельзя добавлять постфактум. Google Ads требует certification, approved countries, responsible-gambling information на лендинге и запрет таргетинга на minors; Meta разрешает online gambling ads только после authorization и при соблюдении применимого законодательства и внутренних правил. В 2026 году Google еще усилил gambling certification requirements, включая требования к policy health и доменам. [2]

Если облачный провайдер **не указан**, то для первой промышленной версии я рекомендую **прямой API от Anthropic** [3] как primary path, а интеграции через Amazon Web Services [4] Bedrock и Google [5] Vertex AI — как secondary deployment options. Причина простая: direct Claude API получает latest features first, умеет Managed Agents, Skills API, Sessions/Environments, и именно там проще всего собрать эталонную архитектуру. Bedrock и Vertex полезны, когда важнее provider-native IAM, procurement, billing, quota governance или внутренняя стандартизация облака, но на сегодня Direct API дает самый полный feature set для твоего кейса. [6]

По моделям базовая стратегия такая: **Claude Opus 4.7** — основной orchestration/architecture/research/review model для сложных задач, агентного кода, compliance reasoning и quality-critical финальных проверок; **Claude Haiku 4.5** — быстрый и дешевый слой для routing, extraction fan-out, classification, deduplication, tag generation, light moderation triage и дешевых субагентов. Anthropic прямо рекомендует Opus 4.7 как стартовую точку для самых сложных задач, а Haiku 4.5 — как fastest / high-volume / latency-sensitive option; при этом для cost-sensitive workflows документация отдельно советует начинать с более быстрого и дешевого класса вроде Haiku 4.5 и апгрейдить только там, где реально нужен capability uplift. [7]

Главная практическая идея плана ниже: сначала зафиксировать **контракты, источники и критерии успеха**, потом поднять **repo policy и Claude Code harness**, затем реализовать **grounded research**, после этого — **generation/review/publish** как отдельные стадии, и только затем — масштабирование, cost controls и multi-agent parallelism. Это соответствует рекомендациям Anthropic: сначала определить success criteria и test/eval loop, использовать structured outputs для parseable outputs, citations для auditable document-grounding, prompt caching/Batch API/Token Counting/Usage & Cost API для операционного контроля, а Claude Code — с CLAUDE.md, path-scoped rules, hooks, sandboxing и subprocess credential scrubbing. [8]

Исходные допущения, ключевые решения и сравнение опций

Исходные допущения

Параметр	Статус	Как трактовать в проекте
Бюджет	**не указано**	Закладывать cost controls с первого дня: routing по моделям, token counting, prompt caching, batch/offline separation, Usage & Cost API
Целевой throughput / QPS / jobs per day	**не указано**	Проектировать очереди, асинхронность, rate-limit aware orchestration и горизонтальное масштабирование с запасом
Предпочитаемый облачный провайдер	**не указано**	Базовая рекомендация: Direct Claude API first; provider-specific hosting — extension path
Основная вертикаль	Широкая content-ops; gambling/affiliate — отдельный high-risk режим	Держать compliance engine channel-aware и jurisdiction-aware
UX	Operator-centric chat + workflow console	Чат не заменяет backend orchestration; он только входная точка

Приоритет источников

Приоритет источников для реализации должен быть таким: **официальная документация Anthropic** по Claude Code, Claude API и Managed Agents; затем **официальные ad policy docs** Google Ads и Meta; затем **официальные docs конкретного облака** по Bedrock/Vertex. Это особенно важно, потому что часть возможностей и ограничений менялась в 2025–2026 годах: Opus 4.7 принес breaking changes в reasoning/sampling behavior, MCP connector имеет platform limitations, а gambling ad policies у Google обновлялись в 2026 году. [9]

Архитектурные опции

Сравнение ниже — это инженерная интерпретация официальных возможностей Anthropic для твоего use case: direct chat/orchestrator work отлично ложится на Messages API, long-running agentic workloads — на Managed Agents, а coding/repo/CI harness — на Claude Code. [10]

Опция	Когда подходит	Плюсы	Минусы	Рекомендация	Основание
Только Messages API	Небольшие deterministic workflows, собственный loop, полный контроль	Максимальный контроль над prompt/tool/state; просто дебажить; удобно для typed services	Ты сам строишь agent loop, среду, tool execution, async orchestration	Хорошо для control-plane, но **недостаточно** как единственный слой	[11]
Только Managed Agents	Долгие асинхронные runs, code/web/file tasks, cloud containers	Built-in agent harness, stateful sessions, event history, environments	Меньше low-level контроля; не все задачи удобно делать только agent-first способом	Хорошо для research/creative/publisher environments, но **не как единственный слой**	[12]
Гибрид: Messages API + Managed Agents + Claude Code	Промышленная content-ops платформа	Лучший баланс: управляемый control plane + long-running workers + repo automation	Чуть сложнее initial architecture	**Рекомендуемый вариант**	[13]

Model tradeoffs

Anthropic рекомендует начинать с Opus 4.7 для самых сложных задач и использовать Haiku 4.5 там, где решают скорость и стоимость. Для этого проекта это означает deliberate routing, а не «одна модель на всё». [14]

Модель	Лучшая роль в платформе	Сильные стороны	Ограничения	Практический вывод	Основание
Claude Opus 4.7	Orchestrator, architecture, hard research, final review, compliance reasoning, coding agent	Самая сильная GA-модель Anthropic; step-change в agentic coding; хорошо для long-horizon work	Non-default `temperature/top_p/top_k` нельзя; adaptive thinking надо включать явно; дороже	Основной интеллект для quality-critical этапов	[15]

Модель	Лучшая роль в платформе	Сильные стороны	Ограничения	Практический вывод	Основание
Claude Haiku 4.5	Routing, extraction, classification, tagging, light QA, fast subagents	Самая быстрая модель, high-volume/latency-sensitive workloads, cost-effective	Меньше reasoning headroom; adaptive thinking нет	Основа cheap fan-out и массовых стадий	[16]
Claude Sonnet 4.6	Опциональный middle tier	Баланс между cost/speed/capability	Не обязателен на первом этапе, если routing уже построен между Opus и Haiku	Можно добавить later как middle lane	[17]

Cloud hosting options

Ниже — сравнение по способу размещения и эксплуатации. Для строки «цена» я намеренно ставлю **«не указано»**, потому что пользователь не задал бюджет, а полная стоимость зависит от egress, network, storage, observability, CI minutes, queue depth, media workloads и режима inference. Официальные model prices есть у Anthropic, но инфраструктурный TCO нужно считать отдельно. [18]

Вариант	Что дает	Ограничения	Когда выбирать	Цена
Direct Claude API / Anthropic-managed	Latest models/features first; Managed Agents; Skills/Agents/Sessions/Environments; полный first-party feature set	Нужно самому встроить в свой cloud perimeter и ops	Базовый и рекомендуемый путь для первой production-версии	**не указано**
AWS Bedrock	Provider-native procurement, IAM, Bedrock ecosystem, Converse/API patterns для Anthropic Claude	Managed Agents first-party only; часть first-party features недоступна напрямую как на Claude API	Если твоя компания стандартизирована на AWS	**не указано**

Вариант	Что дает	Ограничения	Когда выбирать	Цена
Google Vertex AI	Serverless Claude APIs на Agent Platform, provider-native IAM/quotas/observability, streaming, Google ecosystem	Managed Agents first-party only; на Vertex client tools поддерживаются, server tools — нет; MCP connector first-party-only и сейчас не поддержан на Bedrock/Vertex	Если организация стандартизирована на GCP	**не указано**

Содержательно сравнение опирается на официальные документы: Anthropic пишет, что direct API получает latest models/features first и что Managed Agents доступны только через прямой Claude API; AWS документирует работу Anthropic Claude через Messages/Converse patterns; Google документирует Claude на Vertex Agent Platform как fully managed/serverless API и отдельно отмечает поддержку client tools при отсутствии server tools. MCP connector на данный момент не поддержан на Amazon Bedrock и Google Vertex. [19]

Целевая архитектура и операционная модель

Рекомендуемая архитектура — **operator chat + orchestration backend + async workers + grounded docs + compliance gates + publish adapters**. Messages API лучше использовать там, где нужны строгие входы/выходы и свой control loop; Managed Agents — для research-heavy, file/web/code и media-heavy задач в изолированных контейнерах; Claude Code — для построения репозитория, policy-aware implementation и CI workflow automation. Anthropic отдельно описывает Managed Agents как pre-built configurable harness в managed infrastructure для long-running/asynchronous work, environments — как container templates with isolated session instances, а Claude Code — как agentic coding tool, который читает кодовую базу, редактирует файлы и запускает команды. [20]

```
flowchart LR
    U[Operator Chat UI / Console] --> API[Backend API Gateway]
    API --> ORCH[Workflow Orchestrator]
    ORCH --> MQ[Queue / Job Bus]
    ORCH --> MSG[Messages API Services]
    ORCH --> AGT[Managed Agents Sessions]
    ORCH --> DB[(Postgres / Metadata DB)]
    ORCH --> OBJ[(Object Storage)]
    ORCH --> VEC[(Vector / Search Index)]
    ORCH --> OBS[Observability / Metrics / Traces]

    MSG --> TOOLS[Typed Tools / MCP Connectors]
    AGT --> ENV1[research-env]
    AGT --> ENV2[creative-env]
    AGT --> ENV3[publisher-env]

    ENV1 --> DOCS[Docs / PDFs / Policies / Briefs]
    ENV2 --> MEDIA[Media Processing / ffmpeg / Assets]
    ENV3 --> PUB[Approval Gates / Publish Adapters]
```

```
    PUB --> CH1[CMS / Landing Builder]
    PUB --> CH2[Ad Platforms / Internal Queues]
    PUB --> CH3[Archive / Audit Trail]
```

Из этой схемы следуют три явных runtime-контура: `research-env`, `creative-env`, `publisher-env`. Документация Managed Agents прямо говорит, что Environment задает container configuration, может переиспользоваться across sessions, но каждая session получает свой isolated container instance. Это очень хорошо ложится на разделение исследований, креативного производства и публикации/дистрибуции. [21]

```
flowchart TD
    A[Intake brief] --> B[Normalize brief JSON]
    B --> C[Collect documents / sources]
    C --> D[Grounded research pack with citations]
    D --> E[Generate angles / variants]
    E --> F[Brand / factual / policy QA]
    F --> G{Compliant?}
    G -- No --> H[Revise or reject]
    G -- Yes --> I[Prepare publish job]
    I --> J[Human approval gate]
    J --> K{Approved?}
    K -- No --> L[Return to revision]
    K -- Yes --> M[Publish / queue]
    M --> N[Track performance]
    N --> O[Feed evals / prompts / routing]
```

Рекомендуемая repo-структура

Ниже — не официальная, а **рекомендуемая** структура под индустриальный проект. Она сделана так, чтобы Claude Code мог работать через CLAUDE.md, rules, skills и tests, а backend — через typed contracts и repeatable workflows.

```
repo/
  apps/
    operator-ui/
    api/
    worker-messages/
    worker-agents/
    publisher/
  packages/
    schemas/
    prompts/
    workflows/
    tools/
    compliance/
    evals/
    sdk/
  docs/
```

```
    product.md
    architecture.md
    workflows/
    policies/
    runbooks/
  .claude/
    CLAUDE.md
    rules/
    skills/
    agents/
    settings.json
  infra/
    terraform/
    docker/
    ci/
  tests/
    contract/
    integration/
    e2e/
    evals/
  scripts/
  runs/
```

Почему document-grounded и citations — обязательны

Для твоего проекта grounded workflows — не nice-to-have, а базовое требование. Anthropic рекомендует при длинных документах сначала извлекать прямые цитаты, затем делать вывод; для citations нужно передавать документы и включать `citations.enabled=true`; text citations поддерживаются для PDF/plain text/custom content, а citing images from PDFs пока не поддерживается. Это делает research packs и compliance verdicts аудируемыми, что критично в affiliate/gambling контексте. [22]

Пошаговый план по фазам

Ниже — **120 коротких, однозначных и выполнимых шагов**. В каждой фазе: цель, входы, выходы, роли, артефакты, критерии готовности, затем шаги и риски.

Фаза Discovery

Поле	Содержание
Цель	Зафиксировать scope, успех, риски и operating model
Входы	Твои цели, портфолио, список use cases, доступные источники
Выходы	Product brief, success metrics, risk register, workflow map
Роли	Product, Dev, ML Engineer, QA, Legal
Артефакты	`docs/product.md`, `docs/architecture.md`, `docs/workflows/intake.md`, `docs/policies/README.md`

Поле	Содержание
Критерии готовности	Есть список use cases, KPI, red flags и boundaries

Эта фаза должна идти первой, потому что Anthropic рекомендует начинать LLM-продукт с success criteria и evaluations, а не с интерфейса. [23]

D-01. Опиши 10–15 целевых workflow в одном документе без привязки к одной компании.
D-02. Раздели workflow на interactive, async и batch.
D-03. Для каждого workflow выпиши входы, выходы, SLA и human handoff.
D-04. Для каждого workflow выпиши what can go wrong.
D-05. Выдели high-risk режимы: gambling, affiliate, finance-like, health-like.
D-06. Задай KPI: time saved, approval rate, rejection rate, throughput, cost per run.
D-07. Собери 30–50 реальных примеров брифов и документов.
D-08. Собери 20–30 плохих примеров для negative testing.
D-09. Зафиксируй, какие публикации требуют human sign-off.
D-10. Пометь неизвестные параметры как «не указано» и не зашивай guess в архитектуру.

Риск: расплывчатый scope превратит платформу в бесконечный «умный чат». Mitigation: все use cases переводить в workflow cards с measurable definition of done. [23]

Фаза Repo & CLAUDE.md

Поле	Содержание
Цель	Подготовить репозиторий и policy layer для Claude Code
Входы	Product brief, workflow map
Выходы	Инициализированный repo, CLAUDE.md, rules, skills, settings, hooks
Роли	Dev, ML Engineer, DevOps
Артефакты	`.claude/CLAUDE.md`, `.claude/rules/*`, `.claude/settings.json`, `.claude/skills/*`
Критерии готовности	Claude Code запускается предсказуемо и подчиняется policy

Эта фаза опирается на официальные рекомендации: `/init` умеет scaffold CLAUDE.md, skills и hooks; CLAUDE.md стоит держать concise и примерно до 200 строк; path-scoped rules уменьшают шум; hooks дают deterministic control вместо надежды на то, что LLM сама не забудет важный шаг. [24]

R-01. Создай новый monorepo или выбери существующий repo как canonical source.
R-02. Включи `CLAUDE_CODE_NEW_INIT=1`.
R-03. Запусти `/init` и сгенерируй базовые CLAUDE.md, skills и hooks.
R-04. Сократи root `CLAUDE.md` до правил, которые реально нужны на каждой сессии.
R-05. Вынеси длинные инструкции в `@imports` и `.claude/rules/`.
R-06. Добавь path-scoped rules для `apps/api`, `apps/publisher`, `packages/compliance`.
R-07. Создай `CLAUDE.local.md` как gitignored personal layer.

R-08. В `settings.json` включи `defaultMode: "plan"`.
R-09. В deny rules закрой `.env`, `.env.*`, `secrets/**`, production creds.
R-10. Включи sandbox и `failIfUnavailable: true`.

Риск: слишком длинный CLAUDE.md забьет контекст и ухудшит adherence. Mitigation: root file — только устойчивые правила, остальное — rules/skills/imports. [25]

Фаза Schemas & Contracts

Поле	Содержание
Цель	Запереть бизнес-сущности в typed contracts
Входы	Workflow map, sample data
Выходы	JSON Schemas и contract tests
Роли	Dev, ML Engineer, QA, Product
Артефакты	`packages/schemas/*.json`, `tests/contract/*`
Критерии готовности	Любой workflow stage читает/пишет schema-valid JSON

Эта фаза критична, потому что structured outputs дают parseable JSON и strict tool use гарантирует schema-valid tool inputs, что намного надежнее prose-based integration. [26]

S-01. Создай schema `Brief`.
S-02. Создай schema `ResearchSource`.
S-03. Создай schema `ResearchPack`.
S-04. Создай schema `CreativeAngle`.
S-05. Создай schema `CreativeVariant`.
S-06. Создай schema `BrandReviewVerdict`.
S-07. Создай schema `ComplianceVerdict`.
S-08. Создай schema `PublishJob`.
S-09. Создай schema `RunLog`.
S-10. Напиши contract tests на valid/invalid payloads для каждой schema.

Риск: prose drift между этапами. Mitigation: каждое взаимодействие между сервисами — только через schema-valid JSON. [26]

Фаза Dev & CI

Поле	Содержание
Цель	Поднять инженерный контур: lint, tests, CI/CD, PR policy
Входы	Repo, schemas
Выходы	CI pipelines, pre-merge checks, dev environments
Роли	Dev, DevOps, QA
Артефакты	`.github/workflows/*`, `infra/`, `scripts/`

Поле	Содержание
Критерии готовности	Каждый PR гоняет contract/integration/eval subset

Claude Code docs поддерживают GitHub Actions и автоматизацию через @claude/PR flows, но regulated-проекты всё равно требуют собственных checks и human review. [27]

C-01. Подними стандартный lint pipeline.
C-02. Подними unit + contract tests pipeline.
C-03. Добавь integration stage для workflow services.
C-04. Добавь eval subset в pre-merge.
C-05. Добавь artifact upload для failed runs.
C-06. Добавь PR template с секциями “prompt changes / schema changes / policy impact”.
C-07. Добавь branch protection на main.
C-08. Добавь ручной gate для deploy в publisher-related services.
C-09. Настрой Claude Code GitHub Action для feature branches или internal review tasks.
C-10. Зафиксируй release process в `docs/runbooks/release.md`.

Риск: merge без eval awareness. Mitigation: промпты, схемы и compliance logic менять только через PR с contract+eval diff. [28]

Фаза Agent/Workflow Implementation

Поле	Содержание
Цель	Реализовать control plane и runtime workflows
Входы	Schemas, CI, source connectors
Выходы	Operator API, router, workflow engine, tool layer
Роли	Dev, ML Engineer
Артефакты	`apps/api`, `packages/workflows`, `packages/tools`, `packages/prompts`
Критерии готовности	Из одного operator request стартует воспроизводимый workflow run

Messages API предназначен для custom loops, Managed Agents — для long-running asynchronous work; MCP connector умеет direct remote MCP integration из Messages API, но имеет platform limitations и tool-only scope. [29]

A-01. Реализуй operator endpoint `POST /runs`.
A-02. Реализуй workflow router по `intent` и `task_type`.
A-03. Реализуй run state machine: `queued -> running -> review -> approved/rejected -> published`.
A-04. Реализуй typed tool wrapper layer над API/database/storage connectors.
A-05. Реализуй Messages API worker для short deterministic stages.
A-06. Реализуй Managed Agents launcher для deep research и long-running stages.
A-07. Создай agent config `research-agent`.
A-08. Создай agent config `creative-agent`.

A-09. Создай agent config `publisher-agent`.
A-10. Реализуй result normalizer, который приводит outputs к схеме.
A-11. Запиши полный run transcript и metadata в `RunLog`.
A-12. Добавь retry policy и explicit error states вместо silent fallback.

Риск: смешение business logic и prompt logic. Mitigation: маршрутизация и state machine — в коде; LLM — только там, где нужен inference. [30]

Фаза Grounding & Docs

Поле	Содержание
Цель	Построить source-grounded слой для документов и policy-aware research
Входы	PDFs, docs, brand books, offers, policy docs
Выходы	Search/index/doc ingestion, research packs с citations
Роли	Dev, ML Engineer, Product, Legal
Артефакты	`apps/doc-ingest`, `packages/search`, `docs/policies/*`, `storage/documents/*`
Критерии готовности	Любой factual claim в research pack поддержан источником

Anthropic рекомендует для длинных документов quote-first extraction, citations и restriction to provided docs; PDF support и citations документированы отдельно. Для MCP нужно учитывать, что connector работает напрямую из Messages API и сейчас не поддержан на Bedrock/Vertex. [31]

G-01. Определи canonical source classes: brief docs, offer docs, policy docs, competitor docs, historical runs.
G-02. Подними ingestion pipeline для PDF/plain text/custom content.
G-03. Добавь метаданные `source_type`, `jurisdiction`, `channel`, `owner`, `version`.
G-04. Выдели отдельный индекс для policy documents.
G-05. В research workflows включи только document-grounded mode.
G-06. Включи citations на всех документах, где нужен audit trail.
G-07. Реализуй quote extraction до summarization.
G-08. Реализуй source confidence scoring.
G-09. Реализуй stale-source detection по version/date.
G-10. Запрети publish-flow продолжаться, если factual claims не имеют подтверждений.

Риск: research drift в сторону «общих знаний». Mitigation: explicit instruction “use only provided documents”, quote-first, citations-required. [32]

Фаза Generation & QA

Поле	Содержание
Цель	Реализовать controlled generation и многоступенчатый QA
Входы	Brief, ResearchPack, brand rules, policy rules

Поле	Содержание
Выходы	Verified creative variants
Роли	ML Engineer, Dev, QA, Product
Артефакты	`packages/prompts/*`, `packages/compliance/*`, `tests/evals/generation/*`
Критерии готовности	Генерация отделена от review и rejection paths

Prompt best practices у Anthropic рекомендуют clear/direct instructions, sequential steps, XML tags, role prompts, few-shot examples и longform data above the query. Для Opus 4.7 reasoning и tool behavior нужно тюнить через adaptive thinking и effort, а не через устаревшие sampling knobs. [33]

Q-01. Раздели generation, factual review, policy review и brand review на отдельные стадии.
Q-02. Для generation stage используй ResearchPack как единственный factual input.
Q-03. Добавь prompt templates в XML-структуре.
Q-04. Добавь 3–5 few-shot examples для критичных форматов.
Q-05. Добавь post-generation schema validation.
Q-06. Добавь factual claim extraction stage.
Q-07. Добавь citation verification stage.
Q-08. Добавь style/brand consistency stage.
Q-09. Добавь rejection reason taxonomy.
Q-10. Реализуй revise-loop максимум с 2–3 итерациями, затем human escalation.

Риск: генератор начнет писать убедительно, но не аудируемо. Mitigation: generation никогда не минует factual/policy/citation review. [34]

Фаза Publishing & Integrations

Поле	Содержание
Цель	Безопасно публиковать или ставить в очередь только прошедшие артефакты
Входы	Approved variants, compliance verdict, assets
Выходы	Publish jobs, channel adapters, audit trail
Роли	Dev, Product, QA, Legal
Артефакты	`apps/publisher`, `packages/channels`, `docs/policies/channel-rules.md`
Критерии готовности	Ни один publish action не уходит без явного approval gate

Google и Meta рассматривают gambling как restricted category: нужен authorization/certification, approved locations, responsible gambling information и запрет таргетинга на minors; следовательно publish layer обязан быть channel-aware и jurisdiction-aware. [2]

P-01. Определи channel adapters как отдельный слой, не как часть prompt.
P-02. Введи `PublishJob` как обязательный контракт.
P-03. Добавь human approval gate перед любым внешним publish.
P-04. Добавь deny-by-default для regulated channels.
P-05. В `PublishJob` зафиксируй `jurisdiction`, `channel`, `domain`, `decision_id`.
P-06. Реализуй preflight checks по домену, гео и типу оффера.
P-07. Реализуй archive of published payloads и source references.
P-08. Реализуй rollback/archive flow для rejected-after-review артефактов.
P-09. Добавь async delivery queue.
P-10. Добавь reconciliation job между publish outcome и internal status.

Риск: случайная публикация непроверенного regulated content. Mitigation: explicit manual approval + machine-readable preflight + deny-by-default. [35]

Фаза Scaling & Cost

Поле	Содержание
Цель	Снизить стоимость и повысить throughput без потери контроля
Входы	Первые рабочие workflows, реальные run logs
Выходы	Routing, caching, batch mode, queue policies
Роли	ML Engineer, DevOps, Dev
Артефакты	`packages/routing/*`, `docs/costs.md`, `infra/queues/*`
Критерии готовности	Появились explicit cost budgets и model routing rules

API overview и related docs у Anthropic подтверждают, что Message Batches дают 50% cost reduction, prompt caching имеет стандартный 5-minute TTL и optional 1-hour TTL, Token Counting помогает заранее считать токены, а Usage & Cost API дает granular historical usage/cost data. [36]

SC-01. Раздели весь трафик на interactive и offline.
SC-02. Все offline repeatable stages переведи на Message Batches.
SC-03. Подключи Token Counting до отправки больших prompts.
SC-04. Включи 5-minute prompt caching для стабильных prefixes.
SC-05. Включи 1-hour cache только для реально редких, но тяжелых prefixes.
SC-06. Сделай routing: Opus для hard reasoning, Haiku для cheap fan-out.
SC-07. Введи run-level cost budget.
SC-08. Введи per-workflow budget и per-stage budget.
SC-09. Логируй cache hits, misses и batch adoption.
SC-10. Добавь отдельный monthly cost review ritual.

Риск: стоимость «взлетит» не от одной модели, а от плохо спроектированных repeated prefixes и sync traffic. Mitigation: caching, batch/offline split, token preflight, routing by complexity. [37]

Фаза Security & Compliance

Поле	Содержание
Цель	Снизить риск утечек, prompt injection и policy violations
Входы	Repo, secrets model, cloud/network policy, regulated rules
Выходы	Secured settings, secrets boundaries, legal gates
Роли	DevOps, Dev, Legal, QA
Артефакты	`.claude/settings.json`, `docs/security.md`, `docs/policies/*`, hooks
Критерии готовности	Secrets не доступны дочерним процессам; publish paths контролируются

Claude Code поддерживает deny rules, sandboxing с filesystem/network isolation и `CLAUDE_CODE_SUBPROCESS_ENV_SCRUB=1`, который убирает Anthropic/cloud credentials из subprocess environments и на Linux изолирует PID namespace, чтобы bash-процессы не могли читать host process environments через `/proc`. [38]

SE-01. В deny rules закрой чтение `.env`, `.env.*`, `secrets/**`, kube configs и prod creds.
SE-02. Включи `CLAUDE_CODE_SUBPROCESS_ENV_SCRUB=1`.
SE-03. Включи sandboxing как hard gate.
SE-04. Запрети unsandboxed commands, кроме явного allowlist case.
SE-05. Запрети arbitrary `curl` / `wget` из репозитория по умолчанию.
SE-06. Для MCP веди allowlist remote servers.
SE-07. Не подключай недоверенные remote MCP servers.
SE-08. Для gambling/affiliate вынеси legal rules в machine-readable policy pack.
SE-09. Добавь compliance hook, который не даст перейти в publisher stage без verdict.
SE-10. Введи quarterly review для policy docs и domain eligibility assumptions.

Риск: prompt injection через документы или tools приведет к утечке credentials или к illegal publish action. Mitigation: deny rules, env scrub, sandboxing, limited networking, allowlisted MCP, mandatory human gate. [39]

Фаза Monitoring & Eval

Поле	Содержание
Цель	Сделать качество и стоимость измеримыми
Входы	Run logs, outputs, gold datasets
Выходы	Eval suites, dashboards, alerts, retry insights
Роли	QA, ML Engineer, DevOps, Product
Артефакты	`packages/evals/*`, `docs/evals/README.md`, OTEL configs
Критерии готовности	Видны quality, cost, latency, retries, policy fail rates

Anthropic подчеркивает, что LLM applications надо строить через success criteria + evaluations. Claude Code docs поддерживают OTEL-style monitoring, usage/cost/event introspection и hook lifecycle telemetry; Usage & Cost API дает organization-level historical cost/usage. [40]

M-01. Собери gold set для каждого workflow.
M-02. Раздели evals на contract, factual, style, policy, publish-readiness.
M-03. Добавь pairwise eval по старому и новому prompt/version.
M-04. Введи regression gate для prompt changes.
M-05. Подними cost dashboard.
M-06. Подними latency dashboard.
M-07. Подними retry/error dashboard.
M-08. Подними hook execution dashboard.
M-09. Подними model routing dashboard.
M-10. Подними alerting на cost spikes и policy failure spikes.

Риск: система «вроде работает», но деградирует незаметно. Mitigation: versioned evals, dashboards, alerts, release gating на quality и policy metrics. [40]

Фаза Launch & Handover

Поле	Содержание
Цель	Запустить пилот, зафиксировать runbooks и передать владение
Входы	Рабочие workflows, dashboards, approvals
Выходы	Pilot launch, runbooks, ownership matrix
Роли	Product, Dev, ML Engineer, QA, DevOps, Legal
Артефакты	`docs/runbooks/*`, `docs/ownership.md`, `docs/incident-playbook.md`
Критерии готовности	Команда может эксплуатировать систему без автора-одиночки

L-01. Выбери 2–3 workflow для пилота.
L-02. Зафиксируй owner для каждого сервиса и each policy pack.
L-03. Подготовь incident runbook.
L-04. Подготовь rollback runbook.
L-05. Подготовь cost overrun runbook.
L-06. Подготовь QA acceptance checklist.
L-07. Проведи dry run на реальных данных без publish.
L-08. Проведи limited beta с human-only publishing.
L-09. Проанализируй 1–2 недели pilot logs и пересчитай routing.
L-10. Только после успешного пилота включай partial automation на реальный publish queue.

Риск: система остается «в голове у автора». Mitigation: docs, ownership, runbooks, reproducible environments, explicit handover. [41]

Фаза Media processing

Поле	Содержание
Цель	Добавить обработку изображений и видео без разрушения общей архитектуры

Поле	Содержание
Входы	creative workflows, asset specs
Выходы	Media pipeline, ffmpeg jobs, asset manifest
Роли	Dev, ML Engineer, DevOps
Артефакты	`apps/media-worker`, `packages/media/*`, `storage/assets/*`
Критерии готовности	Креативные ассеты создаются и проверяются как отдельные артефакты

Managed Agents environments позволяют задавать container configuration и предустановленные packages, а `creative-env` как раз удобен для системного media layer вроде `ffmpeg`, image tooling и batch asset transforms. Это уже не прямой текст из документации про ffmpeg, а инженерное следствие из environment model. [21]

MD-01. Создай `creative-env` как отдельный container template.
MD-02. Добавь в него `ffmpeg` и минимальный media toolchain.
MD-03. Введи schema `AssetManifest`.
MD-04. Отдели text-generation stage от media-transform stage.
MD-05. Сохраняй все промежуточные ассеты в object storage.
MD-06. Добавь checksum и versioning для каждого asset.
MD-07. Введи safe size/length limits для видео и изображений.
MD-08. Сделай QA на технические параметры ассета отдельно от policy QA.
MD-09. Подготовь batch jobs для массовой перекодировки и ресайза.
MD-10. Архивируй final assets вместе с publish references.

Риск: media pipeline начнет «тащить» за собой всю платформу. Mitigation: media — отдельный worker и отдельный env, а не код внутри operator chat.

Фаза Storage, Queues, Run-state

Поле	Содержание
Цель	Сделать систему надежной под burst и batch нагрузку
Входы	Throughput assumptions, schemas, runtime design
Выходы	Durable storage, queues, idempotent jobs
Роли	DevOps, Dev
Артефакты	`infra/queues/*`, `infra/storage/*`, `packages/state/*`
Критерии готовности	Job можно безопасно повторить, отменить, продолжить и аудировать

ST-01. Выбери metadata DB для runs и artifacts.
ST-02. Выбери object storage для документов и media assets.
ST-03. Введи idempotency keys для всех запусков и publish jobs.
ST-04. Раздели очереди на `interactive`, `deep-research`, `creative-batch`, `publish`.
ST-05. Настрой retry policy per queue.
ST-06. Добавь dead-letter queue.

ST-07. Храни event timeline per run.
ST-08. Делай immutable artifact snapshots для финальных outputs.
ST-09. Введи TTL и archival policy для temporary artifacts.
ST-10. Добавь cancellation и resumption semantics для длительных runs.

Риск: повторные jobs создадут дубли, а failed runs потеряют состояние. Mitigation: idempotency, immutable artifacts, event logs, DLQ.

Практические конфиги, шаблоны и prompt-пакет

Пример `CLAUDE.md`

Этот шаблон соответствует официальным рекомендациям: держать инструкции специфичными и краткими, использовать imports, выносить path-specific правила в `.claude/rules/`, а task-specific heavy workflows — в skills. `CLAUDE.md` — это context, а не enforceable system config, поэтому формулировки должны быть проверяемыми и однозначными. [42]

```
# Product
- This repository implements an AI content-ops platform for industrial
workflows.
- The operator chat is only a control plane over typed workflows.

# Architecture
- Workflow state must be schema-validated JSON.
- Long-running work belongs in Managed Agents sessions.
- Short deterministic work belongs in Messages API workers.
- No publish action may happen without an explicit human approval flag.

# Source grounding
- If a response depends on documents, use grounded extraction with citations.
- If a claim has no supporting quote or citation, remove it or mark it
uncertain.
- Policy and compliance tasks must use quote-first extraction.

# Coding rules
- Reuse existing libraries and patterns.
- Store prompts in /packages/prompts.
- Store JSON schemas in /packages/schemas.
- Store workflow logic in /packages/workflows.
- Never hide business logic inside prompt text.

# Verification
- Run contract tests for schema changes.
- Run focused integration tests for workflow changes.
- Run eval subsets when prompts or compliance logic change.
- Update docs when behavior changes.

# Safety
- Never read .env, .env.*, secrets, or cloud credential files.
- Never call real publish endpoints without approval=true.
```

```
- Treat gambling/affiliate flows as regulated mode.

# Imports
- @docs/product.md
- @docs/architecture.md
- @docs/policies/README.md
- @docs/evals/README.md
```

Пример `settings.json`

Этот пример соединяет permissions, sandboxing и hooks. Документация Claude Code подтверждает поддержку `defaultMode`, `allow/ask/deny`, sandbox hard gate и hooks на lifecycle events, включая `PreToolUse` и `PostToolUse`. [43]

```
{
  "$schema": "https://json.schemastore.org/claude-code-settings.json",
  "permissions": {
    "defaultMode": "plan",
    "allow": [
      "Bash(pnpm lint)",
      "Bash(pnpm test *)",
      "Bash(git status)",
      "Bash(git diff *)",
      "Read(./docs/**)",
      "Read(./packages/**)"
    ],
    "ask": [
      "Bash(git push *)",
      "Bash(terraform apply *)",
      "Bash(kubectl apply *)"
    ],
    "deny": [
      "Read(./.env)",
      "Read(./.env.*)",
      "Read(./secrets/**)",
      "Read(./infra/prod/**)",
      "Bash(curl *)",
      "Bash(wget *)"
    ]
  },
  "sandbox": {
    "enabled": true,
    "failIfUnavailable": true,
    "allowUnsandboxedCommands": false
  },
  "hooks": {
    "PreToolUse": [
      {
        "matcher": "Bash",
        "hooks": [
```

```
          {
            "type": "command",
            "command": "./scripts/guard-bash.sh"
          }
        ]
      }
    ],
    "PostToolUse": [
      {
        "matcher": "Edit|Write",
        "hooks": [
          {
            "type": "command",
            "command": "./scripts/post-edit-check.sh"
          }
        ]
      }
    ],
    "Stop": [
      {
        "matcher": "",
        "hooks": [
          {
            "type": "command",
            "command": "./scripts/summarize-run.sh"
          }
        ]
      }
    ]
  }
}
```

Environment variables, которые стоит задать в managed dev/prod

Переменная `CLAUDE_CODE_SUBPROCESS_ENV_SCRUB=1` — обязательная. Anthropic прямо пишет, что она удаляет Anthropic/cloud credentials из subprocess environments и снижает риск prompt-injection-based secret exfiltration через shell expansion. Для cost-heavy workflows также полезны `ENABLE_PROMPT_CACHING_1H` там, где это действительно оправдано, и OTEL-related vars для мониторинга. [44]

```
export CLAUDE_CODE_SUBPROCESS_ENV_SCRUB=1
export ENABLE_PROMPT_CACHING_1H=1
export OTEL_RESOURCE_ATTRIBUTES="service.name=content-ops,team=ai-platform,env=prod"
```

Готовые prompt templates

Ниже — 7 практических шаблонов. Все они используют то, что Anthropic рекомендует для quality-oriented prompting: clear/direct instructions, XML tags, role setting, few-shot readiness, longform

context above the question, explicit output format, self-check and citations for factual work. Для Opus 4.7 reasoning и thoroughness лучше регулировать `thinking: {type:"adaptive"}` и `output_config.effort`, а не `temperature/top_p/top_k`. [33]

Шаблон оператора: router / operator chat

Когда использовать: первый входной запрос оператора.

Поля:

- `{{user_request}}` — текст оператора
- `{{available_workflows}}` — список workflow slugs
- `{{current_context}}` — уже известные сущности/артефакты

```
<role>
You are the operator workflow router for an AI content-ops platform.
</role>

<context>
Available workflows:
{{available_workflows}}

Current context:
{{current_context}}
</context>

<task>
Read the operator request and decide:
1) which workflow to run,
2) what missing inputs must be collected,
3) whether the task is interactive, async, or batch,
4) whether regulated-mode must be enabled.
</task>

<rules>
- Do not execute the workflow.
- Only route the task and normalize inputs.
- If the request is ambiguous, ask for the minimum missing fields.
- If gambling, affiliate, finance-like, or healthcare-like content is
involved, set regulated_mode=true.
</rules>

<output>
Return only JSON matching the requested schema.
</output>

<request>
{{user_request}}
</request>
```

Expected JSON output:

```
{
  "workflow_slug": "research_pack",
  "task_mode": "async",
  "regulated_mode": true,
  "missing_fields": ["target_geo", "offer_type"],
  "normalized_request": {
    "goal": "Generate a research-backed creative brief",
    "channel": "facebook",
    "vertical": "gambling"
  }
}
```

Шаблон research extraction

Когда использовать: сбор research pack из документов.

Поля:

- `{{documents}}` — документы с source metadata
- `{{research_goal}}` — цель исследования

```
<role>
You are a document-grounded research analyst.
</role>

<documents>
{{documents}}
</documents>

<instructions>
- Use only the provided documents.
- First extract direct quotes that matter.
- Then synthesize findings.
- For every material claim, provide source references.
- If evidence is insufficient, say so explicitly.
</instructions>

<question>
{{research_goal}}
</question>
```

Expected JSON output:

```
{
  "research_goal": "Summarize viable angles for localized casino landing
pages",
```

```
  "findings": [
    {
      "claim": "Brand trust messaging outperformed bonus-only positioning in
prior tests",
      "confidence": "medium",
      "sources": [
        {"source_id": "doc_12", "quote": "Players responded better to trust
and speed than to headline bonus size"}
      ]
    }
  ],
  "unknowns": [
    "No source proves performance for Germany specifically"
  ],
  "recommended_next_queries": [
    "Need updated policy guidance for target jurisdiction"
  ]
}
```

Шаблон brief normalizer

Когда использовать: превращение сырого брифа в typed `Brief`.

Поля:

- `{{raw_brief}}`
- `{{brand_rules}}`

```
<role>
You are a brief normalizer.
</role>

<context>
Brand rules:
{{brand_rules}}
</context>

<task>
Convert the raw brief into a normalized Brief object.
Infer as little as possible.
Mark unknown required fields explicitly.
</task>

<input>
{{raw_brief}}
</input>
```

Expected JSON output:

```
{
  "brief_id": "temp_001",
  "goal": "Produce 12 localized ad-copy variants",
  "channel": "facebook",
  "geo": null,
  "language": "ru",
  "vertical": "affiliate",
  "constraints": [
    "No unsupported factual claims",
    "No publication without approval"
  ],
  "required_unknowns": ["geo", "offer_name", "landing_domain"]
}
```

Шаблон generation template

Когда использовать: controlled generation of variants.

Поля:

- `{{brief}}`
- `{{research_pack}}`
- `{{brand_rules}}`
- `{{target_count}}`

```
<role>
You are a creative generation engine for industrial content operations.
</role>

<context>
Brief:
{{brief}}

Research pack:
{{research_pack}}

Brand rules:
{{brand_rules}}
</context>

<task>
Generate {{target_count}} creative variants.
Use only facts supported by the research pack.
Each variant must be materially distinct in angle, not just wording.
</task>

<rules>
- No unsupported claims.
- No policy-sensitive wording if evidence is weak.
```

```
- Keep each variant concise and production-ready.
</rules>
```

Expected JSON output:

```
{
  "variants": [
    {
      "variant_id": "cv_001",
      "angle": "trust_and_speed",
      "headline": "Быстрый доступ и понятный старт",
      "body": "Без перегруза, с акцентом на удобство и понятный путь
пользователя.",
      "factual_dependencies": ["rp_claim_04"],
      "risk_flags": []
    }
  ]
}
```

Шаблон compliance check

Когда использовать: отдельная policy/regulated review стадия.

Поля:

- `{{channel}}`
- `{{jurisdiction}}`
- `{{policy_pack}}`
- `{{candidate_assets}}`

```
<role>
You are a regulated-content compliance reviewer.
</role>

<context>
Channel: {{channel}}
Jurisdiction: {{jurisdiction}}
Policy pack:
{{policy_pack}}
Candidate assets:
{{candidate_assets}}
</context>

<task>
Evaluate whether the assets are publishable for this channel and
jurisdiction.
Return a strict verdict.
</task>
```

```
<rules>
- Separate policy violations from uncertainty.
- If a required condition is unknown, return needs_human_review.
- Do not rewrite assets. Only review them.
</rules>
```

Expected JSON output:

```
{
  "verdict": "needs_human_review",
  "publishable": false,
  "violations": [],
  "uncertainties": [
    "Landing domain ownership not verified",
    "Responsible gambling disclosure not confirmed"
  ],
  "required_actions": [
    "Verify domain control",
    "Verify responsible-gambling section on landing page"
  ]
}
```

Шаблон publish job preparation

Когда использовать: перед постановкой в очередь на публикацию.

Поля:

- `{{approved_variant}}`
- `{{compliance_verdict}}`
- `{{channel_config}}`
- `{{asset_manifest}}`

```
<role>
You are a publish job compiler.
</role>

<context>
Approved variant:
{{approved_variant}}

Compliance verdict:
{{compliance_verdict}}

Channel config:
{{channel_config}}

Asset manifest:
{{asset_manifest}}
```

```
</context>

<task>
Compile a publish-ready job object.
Do not publish.
</task>

<rules>
- If approval is missing, stop.
- If compliance verdict is not publishable=true, stop.
- Output must be complete and machine-readable.
</rules>
```

Expected JSON output:

```
{
  "publish_job_id": "pj_2026_04_24_001",
  "channel": "facebook",
  "jurisdiction": "PL",
  "approval_required": true,
  "approval_present": true,
  "payload": {
    "headline": "Быстрый и понятный старт",
    "body": "Контролируемый и проверенный текст",
    "landing_domain": "example.com"
  },
  "status": "ready_for_queue"
}
```

Шаблон factual self-check

Когда использовать: перед финальным human review.

Поля:

- `{{output_text}}`
- `{{research_pack}}`

```
<role>
You are a factual verifier.
</role>

<context>
Research pack:
{{research_pack}}
</context>

<task>
Extract every factual claim from the output.
```

```
For each claim, find supporting evidence in the research pack.
If no support exists, mark the claim unsupported.
</task>

<input>
{{output_text}}
</input>
```

Expected JSON output:

```
{
  "claims": [
    {
      "claim_text": "Пользователи лучше реагируют на trust messaging",
      "supported": true,
      "evidence_refs": ["rp_claim_02"]
    },
    {
      "claim_text": "Этот подход лучше работает в Польше",
      "supported": false,
      "evidence_refs": []
    }
  ],
  "overall_status": "revise_required"
}
```

Шаблон batch triage

Когда использовать: массовая сортировка брифов и задач.

Поля:

- `{{items}}`

```
<role>
You are a batch triage classifier.
</role>

<task>
Classify each item by workflow, risk level, model tier, and processing mode.
Keep reasoning internal and return JSON only.
</task>

<input>
{{items}}
</input>
```

Expected JSON output:

```
{
  "items": [
    {
      "id": "req_001",
      "workflow": "brief_normalization",
      "risk_level": "low",
      "model_tier": "haiku",
      "mode": "batch"
    },
    {
      "id": "req_002",
      "workflow": "compliance_review",
      "risk_level": "high",
      "model_tier": "opus",
      "mode": "async"
    }
  ]
}
```

Рекомендованные runtime-настройки моделей

Для Opus 4.7 Anthropic рекомендует adaptive thinking и effort-based steering; migration guide прямо говорит, что `thinking: {type:"enabled", budget_tokens:N}` больше не поддерживается, а non-default `temperature/top_p/top_k` приведут к ошибке. Поэтому production defaults для quality-critical routes лучше задавать так: `thinking: {type:"adaptive"}`, `output_config.effort: "high"` или `"xhigh"`, а если reasoning progress нужен в UI — включать `thinking.display: "summarized"`. Для Haiku 4.5 лучше держать минимальную prompt complexity и использовать его как fast worker, а не как финального policy arbiter. [45]

```
{
  "model": "claude-opus-4-7",
  "thinking": {
    "type": "adaptive",
    "display": "summarized"
  },
  "output_config": {
    "effort": "xhigh"
  },
  "max_tokens": 64000
}
```

Риски, production readiness и release checklist

Риски и mitigation по всему проекту

Самый большой риск — сделать систему «слишком чатовой» и слишком мало workflow-driven. Это обычно приводит к непредсказуемым side effects, плохой трассировке и быстрому росту стоимости. Противоядие: state machine, typed schemas, run logs, approval gates и separation of

concerns между routing, research, generation, QA и publishing. Это не прямой текст из документации, а engineering conclusion, опирающийся на structured outputs, success/evals и разделение Messages API vs Managed Agents. [46]

Второй риск — prompt sprawl. Если промпты разъедутся между кодом, markdown, личной памятью и скрытыми чат-сессиями, ты потеряешь повторяемость. Противоядие: prompts only in versioned files, CLAUDE.md короткий, rules path-scoped, heavy workflows — в skills, prompt changes — через PR + eval diff. [47]

Третий риск — security/compliance drift. MCP и tools резко увеличивают power surface, а gambling/affiliate требует channel/jurisdiction-aware gates. Противоядие: allowlisted MCP, deny secrets, env scrub, sandboxing, policy pack versioning, human publish approvals, quarterly policy review. Anthropic отдельно предупреждает о trust requirements для MCP connector, а Google/Meta policies делают regulated publishing materially risky. [48]

Production readiness criteria

Проект можно считать production-ready, когда одновременно выполнены следующие условия:

1. Каждый workflow пишет и читает schema-valid JSON.
2. Каждый factual workflow умеет document-grounded mode и citations.
3. Каждый regulated workflow имеет отдельный `ComplianceVerdict`.
4. Ни один внешний publish не происходит без machine gate и human approval.
5. Есть eval suite и regression gate для prompts, schemas и compliance rules.
6. Есть cost visibility: token counting, Usage & Cost API, cache stats, batch stats.
7. Есть security controls: deny rules, sandboxing, env scrub, MCP allowlists.
8. Есть runbooks, ownership, incident handling и rollback path.

Эти критерии напрямую вытекают из официальных возможностей Anthropic по structured outputs, citations, evals, cost/usage tooling, Claude Code settings/sandboxing/hooks и Managed Agents environments. [49]

Краткий чеклист релиза

- [ ] `CLAUDE.md`, `.claude/rules/`, `.claude/settings.json` зафиксированы и отревьюены.
- [ ] `CLAUDE_CODE_SUBPROCESS_ENV_SCRUB=1` включен.
- [ ] Sandboxing включен как hard gate.
- [ ] Схемы `Brief`, `ResearchPack`, `CreativeVariant`, `ComplianceVerdict`, `PublishJob` стабилизированы.
- [ ] Все prompt templates лежат в version control.
- [ ] Research workflows используют doc-grounding и citations.
- [ ] Policy pack для targeted channels/jurisdictions актуален.
- [ ] Batch API, prompt caching и model routing включены там, где это оправдано.
- [ ] Usage & Cost dashboards и alerts работают.
- [ ] Publish adapters выключены по умолчанию до approval gate.
- [ ] Pilot прогнан на реальных данных без автопубликации.
- [ ] Runbooks и ownership matrix оформлены.

Open questions / limitations

Ниже — вопросы, которые я **не заполнял догадками**, потому что они у тебя не указаны и реально меняют инженерные решения:

- бюджет проекта — **не указано**;
- требуемый throughput / суточный объем runs — **не указано**;
- предпочитаемый облачный провайдер — **не указано**;
- планируемые каналы дистрибуции кроме общих контент-ops сценариев — частично **не указано**;
- какие именно media-generation providers будут участвовать в video/image pipeline — **не указано**.

Из-за этого я дал **намеренно провайдер-агностичный, но production-oriented план**, где direct Anthropic path — базовая рекомендация, а provider-specific развилки обозначены как варианты.

Карта основных источников

Основные источники, на которых основан этот план:

- Claude API overview, Messages API, Message Batches, Token Counting, partner availability, direct-vs-managed split. [50]
- Claude Managed Agents overview, environments, stateful sessions, event history. [51]
- Claude Code overview, memory/CLAUDE.md, rules, settings, hooks, sandboxing, env vars, subagents, skills, GitHub Actions, monitoring. [52]
- Prompt engineering, long-context prompting, XML structure, few-shot guidance, Opus 4.7 migration and settings. [33]
- Structured outputs, citations, PDF support, eval methodology, hallucination reduction. [53]
- Prompt caching, Usage & Cost API, Token Counting, pricing/cost-control primitives. [54]
- MCP connector and MCP standard. [55]
- Google Ads gambling policy и certification updates 2026. [56]
- Meta online gambling/gaming ad authorization rules. [57]
- Claude on Vertex AI и Anthropic Claude on AWS Bedrock docs. [58]

---

[1] [10] [11] [12] [13] [20] [29] [30] [51] Claude Managed Agents overview - Claude API Docs
https://platform.claude.com/docs/en/managed-agents/overview

[2] [4] [35] [56] Apply to advertise certain products and services using Google Ads - Advertising Policies Help
https://support.google.com/adspolicy/answer/16114090?hl=en-IE

[3] [24] [25] [41] [42] [47] https://code.claude.com/docs/en/memory
https://code.claude.com/docs/en/memory

[5] [6] [19] [36] [37] [50] https://platform.claude.com/docs/en/api/overview
https://platform.claude.com/docs/en/api/overview

[7] [14] [15] [16] Models overview - Claude API Docs
https://platform.claude.com/docs/en/about-claude/models/overview

[8] [23] [40] Define success criteria and build evaluations - Claude API Docs
https://platform.claude.com/docs/en/test-and-evaluate/develop-tests

9 45 https://platform.claude.com/docs/en/about-claude/models/migration-guide
https://platform.claude.com/docs/en/about-claude/models/migration-guide

17 https://platform.claude.com/docs/es/home
https://platform.claude.com/docs/es/home

18 https://platform.claude.com/docs/en/about-claude/pricing
https://platform.claude.com/docs/en/about-claude/pricing

21 Cloud environment setup - Claude API Docs
https://platform.claude.com/docs/en/managed-agents/environments

22 31 32 34 https://platform.claude.com/docs/en/test-and-evaluate/strengthen-guardrails/reduce-hallucinations
https://platform.claude.com/docs/en/test-and-evaluate/strengthen-guardrails/reduce-hallucinations

26 46 49 53 Structured outputs - Claude API Docs
https://platform.claude.com/docs/en/build-with-claude/structured-outputs

27 28 https://code.claude.com/docs/en/github-actions
https://code.claude.com/docs/en/github-actions

33 https://platform.claude.com/docs/en/build-with-claude/prompt-engineering/claude-prompting-best-practices
https://platform.claude.com/docs/en/build-with-claude/prompt-engineering/claude-prompting-best-practices

38 43 https://code.claude.com/docs/en/settings
https://code.claude.com/docs/en/settings

39 44 Environment variables - Claude Code Docs
https://code.claude.com/docs/en/env-vars

48 55 https://platform.claude.com/docs/en/agents-and-tools/mcp-connector
https://platform.claude.com/docs/en/agents-and-tools/mcp-connector

52 Claude Code overview - Claude Code Docs
https://code.claude.com/docs/en/overview

54 https://platform.claude.com/docs/en/build-with-claude/prompt-caching?38d7aa68_page=7&cc61befa_page=2&fcdaa149_page=2
https://platform.claude.com/docs/en/build-with-claude/prompt-caching?38d7aa68_page=7&cc61befa_page=2&fcdaa149_page=2

57 Online Gambling and Games | Transparency Center - Meta
https://transparency.meta.com/policies/ad-standards/restricted-goods-services/gambling-games/?utm_source=chatgpt.com

58 Anthropic's Claude models | Generative AI on Vertex AI | Google Cloud Documentation
https://docs.cloud.google.com/vertex-ai/generative-ai/docs/partner-models/claude